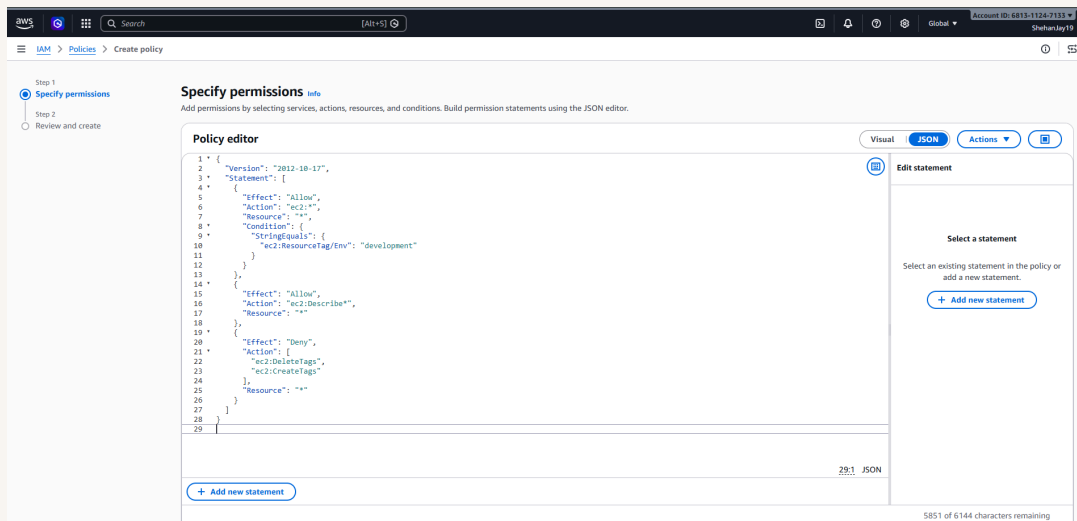




Cloud Security with AWS IAM



Shehan Tharushe





Introducing Today's Project!

Project overview

In this project, I will demonstrate how to host a static website on Amazon S3 by creating a bucket, uploading HTML and image files, configuring static website hosting, and making the website publicly accessible using bucket policies and Access Control Lists. I am doing this project to learn the core features of Amazon S3 as part of project two of the AWS Beginners Challenge with NextWork.ai, where I am building my cloud engineering skills from scratch and creating a hands-on portfolio that showcases my ability to work with real AWS services

Tools and concepts

"Services I used were Amazon EC2 to launch and manage virtual machine instances in the cloud, and AWS Identity and Access Management to create and manage secure access controls for those instances. Key concepts I learnt include how to launch EC2 instances and assign tags such as Name and Env to identify and organize them, how IAM policies use Effect, Action, and Resource in JSON format to define what actions are allowed or denied on specific resources, how to attach IAM policies to user groups so that all users in the group automatically inherit the same permissions, how to create IAM users with their own login credentials so that team members like an intern can access the AWS account without sharing the root user details, how to create an account alias to simplify the AWS login URL, and how the principle of least privilege works in practice by allowing the intern full access to the development instance while completely blocking access to the production instance.

Project reflection

This project took me approximately one hour to complete from launching the EC2 instances to successfully testing the IAM policy with the intern user. The most challenging part was writing and understanding the JSON IAM policy, specifically figuring out how the Effect, Action, Resource, and Condition elements



It was most rewarding to see the policy working correctly in real time, where the intern user was blocked from stopping the production instance with a red error message and successfully able to stop the development instance with a green success message, proving that the IAM permissions were set up exactly as intended



Tags

What I did in this step

In this step, I will launch two Amazon EC2 because by doing that we can increase computation power

Understanding tags

Tags in AWS are labels made up of a key and a value pair that you can attach to any AWS resource such as an EC2 instance, S3 bucket, or any other service. For example, a tag could have a key of Name and a value of my-web-server, or a key of Project and a value of NextWork. Tags are useful for organizing and identifying your resources, especially when you have many of them running at the same time. They also help with tracking costs in your billing dashboard by showing you how much each tagged resource is spending, and make it easier to search and filter resources in the AWS Console

My tag configuration

The tags I have used on my EC2 instances are Name and Env. The value I assigned for the Name tag is nextwork-dev-shehan which identifies the instance by project and my name, and the value I assigned for the Env tag is development which indicates that this instance is being used for a development environment rather than a production one.



aws

Search [Alt+S]

United States (O)

Account ID: 6813-1124-7133

ShehanJay19

EC2

Instances

Launch an instance

Launch an instance

Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

▼ Name and tags

Info

Key

Info

Q Name

X

Value

Info

Q network-dev-sheh

X

Resource types

Info

Select resource types

▼

Instances

X

Remove

Key

Info

Q Env

X

Value

Info

Q development

X

Resource types

Info

Select resource types

▼

Instances

X

Remove

Add new tag

You can add up to 48 more tags.



IAM Policies

What I did in this step

In this step, I will create an IAM policy because we want to give development instance permission to new intern

Understanding IAM policies

IAM Policies are rules that define who has permission to access AWS resources and what actions they are allowed to perform on them. For example, an IAM Policy can allow a user to only read files in an S3 bucket but not delete them, or allow access to EC2 instances but not billing information. These rules are written in JSON format and are attached to IAM users, groups, or roles to control their level of access across AWS services

The policy I set up

For this project, I've set up a policy using JSON

Policy effect

I have created a policy that allows `ec2:*` which means full access to all EC2 actions, but only on resources that have the `Env` tag set to `development`. This means the user can only interact with EC2 instances that are tagged as development environments, such as my `nextwork-dev-shehan` instance. The policy also allows `ec2:Describe*` which lets the user view and describe all EC2 resources across any environment without being able to change them.



Finally, the policy has a Deny effect on `ec2:DeleteTags` and `ec2:CreateTags` which prevents the user from adding or removing tags on any EC2 resource, ensuring that no one can change the Env tag to gain access to instances they are not supposed to manage.

Understanding Effect, Action, and Resource

In a JSON policy, Effect defines whether the rule is allowing or denying access, and it can only have two values which are Allow or Deny. Action defines what specific AWS actions or operations the rule applies to, for example `ec2:*` means all EC2 actions while `ec2:Describe*` means only describe or view actions. Resource defines which specific AWS resources the rule applies to, and when it is set to an asterisk it means the rule applies to all resources. Together these three elements work as a complete rule that tells AWS who can do what and on which resources.



My JSON Policy

The screenshot shows the AWS IAM console interface for creating a new policy. The page is titled "Specify permissions" and includes a sub-header "Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor." The left sidebar shows the navigation menu with "IAM" selected, and the "Specify permissions" step is highlighted. The main content area is the "Policy editor" which is in "JSON" mode. The JSON editor displays a policy document with the following content:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "ec2:*",
7       "Resource": "*",
8       "Condition": {
9         "StringEquals": {
10          "ec2:ResourceTag/Env": "development"
11        }
12      }
13    },
14    {
15      "Effect": "Allow",
16      "Action": "ec2:Describe*",
17      "Resource": "*"
18    },
19    {
20      "Effect": "Deny",
21      "Action": [
22        "ec2:DeleteTags",
23        "ec2:createTags"
24      ],
25      "Resource": "*"
26    }
27  ]
28 }
29
```

Below the JSON editor, there is a button labeled "+ Add new statement". To the right of the JSON editor, there is a panel titled "Edit statement" which contains a "Select a statement" section with a button labeled "+ Add new statement". At the bottom right of the JSON editor, there is a status bar indicating "29/1 JSON" and "3851 of 6144 characters remaining".



Account Alias

What I did in this step

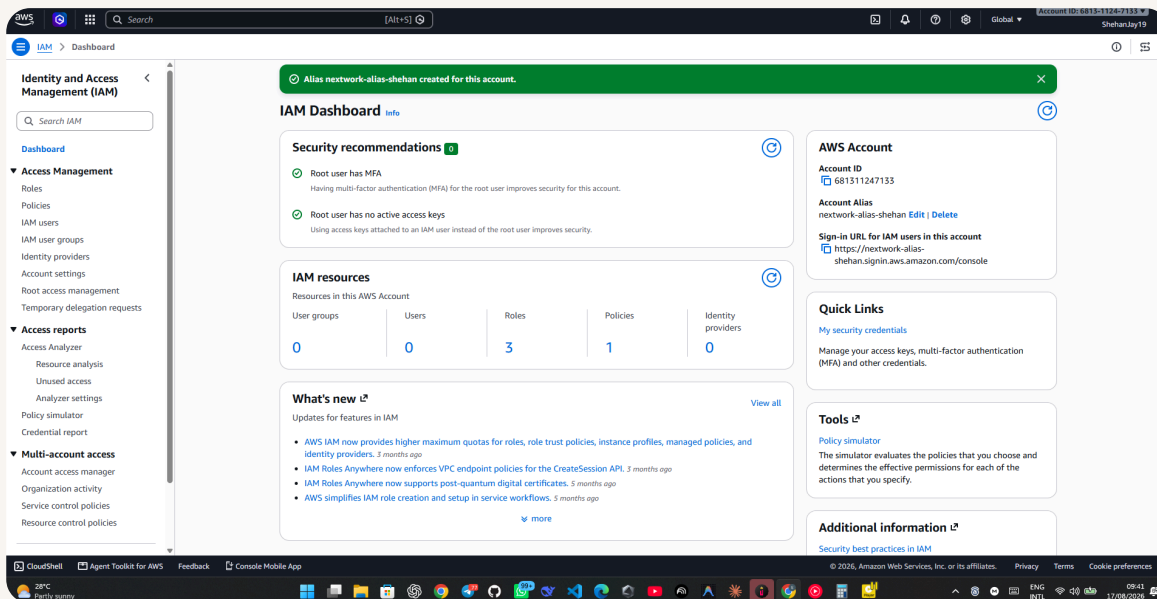
In this step, I will create an Account Alias to simplify the login URL for my AWS account, because by default AWS gives every account a long and difficult to remember login URL that includes a random account ID number. By setting up an Account Alias I can replace that complicated URL with a short and easy to remember name, making it much simpler for the intern to find and access the AWS login page without needing to remember or share the full account ID

Understanding account aliases

An account alias is another that person that have permission to acces the AWS servers

Setting up my account alias

Creating an account alias took me like 3 min Now, my new AWS console sign-in URL is `https://nextwork-alias-*****.signin.aws.amazon.com/console`





IAM Users and User Groups

What I did in this step

"In this step, I will create an IAM group and an IAM user for the intern using AWS Identity and Access Management, because sharing my own AWS account details with the intern would give them access to everything including the production instance which they are not supposed to manage. By creating a separate IAM user the intern gets their own unique username and password to log in, and by adding them to an IAM group with the development policy attached, they will only have access to the development instance and nothing else. This is the safest and most professional way to give someone access to an AWS account without compromising security or exposing resources they should not have access to

Understanding user groups

IAM user groups are collections of IAM users that share the same permissions and policies. Instead of attaching a policy to each user one by one, you can attach a policy to a group and every user added to that group automatically inherits those permissions. For example, if I create a group called developers and attach the development policy to it, any IAM user added to that group will automatically only have access to the development EC2 instance. This makes it much easier to manage permissions across multiple users, especially in a team environment where different people need different levels of access

Attaching policies to user groups

I attached the policy I created to this user group, which means any IAM user added to this group will automatically inherit the same permissions defined in the policy. In this case the intern, who is a member of the group, will only be able to perform actions on EC2 instances that are tagged with Env set to development, can view and describe all EC2 resources but not change them, and cannot add or remove tags on any instance.



This ensures that the intern has just enough access to do their job on the development instance without being able to touch the production instance or any other resources in the AWS account.

Understanding IAM users

IAM users are individual accounts created inside an AWS account that have their own unique username and password. Each IAM user can be given specific permissions to control what they can and cannot do in AWS



Logging in as an IAM User

Sharing sign-in details

The first way is to download a CSV file that contains the user's login credentials including their username, password, and the account sign in URL, and then share that file directly with the user. The second way is to email the sign in instructions directly to the user through the AWS Console, which sends them their username, temporary password, and the account alias login URL they need to access the AWS account.

Observations from the IAM user dashboard

Once I logged in as my IAM user, I noticed that the dashboard was showing several error messages and permission warnings across many AWS services, and that the intern could only see limited information compared to the root user account. This was because the IAM user has only been given specific permissions through the policy attached to the user group, meaning they do not have access to view or manage any services outside of the development EC2 instance. This is exactly what we expected and confirms that the principle of least privilege is working correctly, where users are only given the minimum access they need to do their job and nothing more.



aws

Search

Ask Amazon

Account ID: C8151122-2135-4 ShehanJay19

[IAM](#) > [IAM users](#) > Create user

User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

Console sign-in details

Email sign-in instructions

Console sign-in URL

https://nextwork-alias-shehan.signin.aws.amazon.com/console

User name

nextwork-dev-amaya

Console password

Show

Cancel

Download .csv file

Return to users list



Testing IAM Policies

What I did in this step

In this step, I will test the intern's IAM user access by logging into the AWS Console using their credentials and attempting to access both the development and production EC2 instances, because before handing over the login details to the intern it is important to verify that the permissions are working exactly as expected. By testing the access myself I can confirm that the intern can successfully perform actions on the development instance tagged with Env set to development, and that they are correctly blocked from accessing the production instance, ensuring that the IAM policy, user group, and user have all been set up correctly and securely.

Testing policy actions

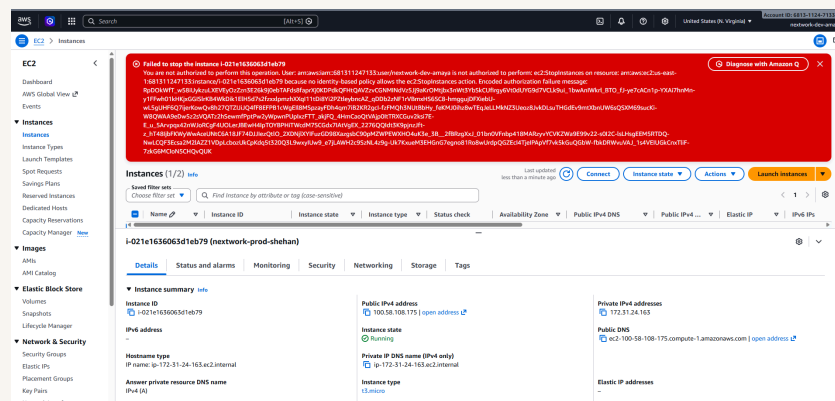
I tested my JSON IAM policy by attempting to stop both the development instance named nextwork-dev-shehan and the production instance named nextwork-prod-shehan while logged in as the intern IAM user. When I tried to stop the production instance I received an authorization error confirming that the IAM user did not have permission to perform that action. However when I tried to stop the development instance named nextwork-dev-shehan with the Env tag set to development, it successfully initiated the stopping process as shown by the green success message and the instance state changing to Stopping. This confirmed that my IAM policy was working correctly by allowing full access to the development instance while blocking any actions on the production instance.

Stopping the production instance

When I tried to stop the production instance named nextwork-prod-shehan, I received an error message saying I was not authorized to perform this operation. This was because the IAM policy attached to my user group only allows full EC2 access on instances tagged with Env set to development, and since the production instance does not have that tag the policy correctly blocked the



This confirms that the IAM permissions are working exactly as intended, where the intern can only manage the development instance and is completely prevented from stopping, modifying, or interfering with the production instance.



Stopping the development instance

"Next, when I tried to stop the development instance named nextwork-dev-shehan, I received a green success message saying the stopping was successfully initiated and the instance state changed to Stopping. This was because the IAM policy attached to the intern's user group allows full EC2 access on instances that have the Env tag set to development, and since the development instance had that exact tag the policy correctly permitted the action, confirming that the intern has the right level of access to manage the development instance as intended.



A screenshot of the AWS Management Console showing the EC2 Instances page. The console is displaying a list of instances, with one instance, 'network-dev-...', in the 'stopping' state. A notification banner at the top indicates 'Successfully initiated stopping of i-095066f822209454c'. The left sidebar shows the navigation menu with categories like EC2, Images, Elastic Block Store, and Network & Security. The main content area shows the details for the selected instance, including its ID, name, state, type, and various IP addresses.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv4 IPs
network-dev-...	i-095066f822209454c	stopping	t3.micro	initializing	us-east-1c	ec2-100-58-708-175.us...	100.58.108.175	-	-

I-095066f822209454c (network-dev-shehan)

Instance summary

Instance ID
i-095066f822209454c

Public IPv4 address
100.27.231.17 [open address](#)

Private IPv4 addresses
172.31.16.105

IPv4 address
-

Instance state
stopping

Public DNS
ec2-100-27-231-17.compute-1.amazonaws.com [open address](#)

Hostname type
IP name: ip-172-31-16-105.ec2.internal

Private IP DNS name (IPv4 only)
ip-172-31-16-105.ec2.internal

Answer private resource DNS name
IPv4 (A)

Instance type
t3.micro

Elastic IP addresses
-



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

